MGT 634 Quiz Study Guide (Thu Oct 15, 2026)

Built from every class note and slide deck (classes 1–12; §9 updated Oct 13), the syllabus, the course readings and the two practice papers (Fall 2025 practice quiz and Fall 2025 quiz). Written so you can learn each idea from zero: every term is explained the first time it appears.

---

0. What the quiz is and how to prepare

Format (both past papers are identical in structure).

Section	What it looks like	Points
1	Six **True / False / Uncertain** statements. You pick T, F or U **and explain why**.	6 × 5 = 30
2	An **economics paper's regression table**: interpret the coefficients, do a small calculation.	10
3	A **big-data workflow** proposed by a colleague or intern: find the problems, fix them, spot a data-quality issue.	10
	Total, 80 minutes, closed book, **no calculator**, no AI, no coding required.	60

What the syllabus says it tests: "all the material from the first half of the course … the exam will not require coding, but you will need to demonstrate a strong understanding of the empirical methods, analytical concepts, and practical insights … clearly explain methodological choices, interpret empirical results, and apply key economic concepts."

What counts as "first half": classes 1–12 (Sep 3 – Oct 13). Classes 11 (Oct 8, data collection) and 12 (Oct 13, data processing) haven't happened yet. §9 previews them, and you should add your own notes after each class.

Watch out: last year's papers include two topics we **have not covered yet** this year: supply-and-demand **instrumental variables** (syllabus class 15, Oct 29) and **discrete-choice / willingness-to-pay** models (class 18, Nov 10). Both papers have a supply/demand instrument question, so section 11 gives you a short "insurance" explanation. The regression-table question can still be answered with what we know: reading coefficients, signs, units and ratios.

How to answer a True / False / Uncertain question (5 points each)

1. **Commit**: write T, F or U first.
2. **Name the concept** being tested, in its proper words (e.g. "this is about omitted variable bias").
3. **Explain the mechanism in 2–4 sentences**, ideally with a tiny example or the formula.
4. **Say what would make it true** (or under what condition the answer flips). This is where "Uncertain" lives. Graders reward the reasoning, not the letter.

Most statements in these papers are **False**, because they contain one over-strong word: *always*, *must*, *definitely*, *exactly*, *cannot*, *therefore*. Hunt for that word.

Study plan (8 days)

Day	Date	Do this (about 1.5–2 h a day)
1	Wed Oct 7	§1 (computing, pandas) and §2 (missing data, outliers). Self-test questions 7–11.
2	Thu Oct 8	Class 11 (data collection). Afterwards: §3 (visualization) + §9 preview; add your class notes to §9.
3	Fri Oct 9	**Regression day**: §5 in full. Write out OLS, R^2, omitted variable bias and fixed effects from memory.
4	Sat Oct 10	§4 (merges, big data) + §7 (workflows). Redo practice Section 2 and 2025 final Section 3 **without looking** at the answers.

Day	Date	Do this (about 1.5–2 h a day)
5	Sun Oct 11	§6 (validation, the free-delivery case) + §8 readings + §10 (later topics). Self-test bank (§12), first pass.
6	Mon Oct 12	**Full timed mock**: practice quiz, 80 minutes, closed book. Then mark it with §11 and §10.2.
7	Tue Oct 13	Class 12 (data processing) + HW2 due. Add notes to §9. Second timed mock: the 2025 final.
8	Wed Oct 14	Light review: §13 one-page summary, §11 table, and any self-test questions you missed. Sleep.

---

1. The computing environment (classes 1–3)

HPC (high-performance computing) cluster. A large shared computer in the SOM basement (~3,000 processors, 200 petabytes of storage). Our data is too big for a laptop, so we run code on it.

- **Open OnDemand (OOD)**: the web page you log into to use the cluster (needs Yale Secure WiFi or the VPN; use a private/incognito window). From it you launch **JupyterLab**, the notebook where you write and run Python.
- **Slurm**: the scheduler. You ask for resources (e.g. 4 cores, 32 GB memory, 2 hours) and it starts your job when they're free. Don't over-request, and close jobs when done.
- **Cores vs memory (RAM)**: cores = how many things run at once (speed); RAM = how much data fits in working memory at once. Loading data bigger than RAM **crashes** the program ("kernel died", "MemoryError"). In class a 5 GB file crashed a 4 GB session.
- **Terminal commands** (not tested heavily, but know them): `pwd` (where am I), `ls` (list files), `cd ~` (go home), `cd ..` (go up one folder), `top` (live view of running processes).

File system and permissions.

- **Home directory** (`~`, e.g. `/home/netid`): you can read and write here. Save your code and outputs here.
- **Course/project folder** (`/gpfs/project/esai_2026`): **read-only**. You can read the data but not change or save files there. That's why raw data stays untouched.
- **Symlink**: a shortcut. `~/esai_2026` points to the project folder, so the data looks like it's in your home folder without being copied.
- **Absolute path** = the full address from the root (`/gpfs/project/esai_2026/data/file.csv`); always works. **Relative path** = the address from where you are now (`data/file.csv`); shorter and portable, but breaks if you run from a different folder. The professor recommends absolute paths in class. In shared projects, build paths from a variable (e.g. your NetID) so teammates can reuse the code.

Python basics.

- A script has three parts: **imports** (libraries such as `pandas`), **definitions** (your functions, `def add(a, b): return a + b`), **execution** (running things).
- Installing a package on the cluster: `!{sys.executable} -m pip install --user pandas`. `--user` installs into your home folder, because you can't write anywhere else.
- **Smart quotes trap**: copying code from slides gives curly quotes, which Python rejects with a `SyntaxError`. Retype them.

pandas essentials (pandas = the Python library for tables, called DataFrames).

- `pd.read_csv(path, nrows=1000)`: read only the first 1,000 rows (test on a sample first). `usecols=[...]`: read only the columns you need (saves memory).

- `df.head()`, `df.columns`, `df.iloc[0]` (first row by position), `df.describe()` (count, mean, standard deviation, min, quartiles, max: the fastest way to spot strange values).
- Columns with spaces need brackets: `df["Trip Total"]`, not `df.Trip Total`.
- New column: `df["mph"] = df["miles"] / (df["seconds"] / 3600)`. Dividing by zero gives `inf`. That's a **data problem** (zero-duration trips), not a code bug.
- **Filtering rows**: `df.loc[df["salary"] > 100000]` keeps only the rows where the condition is true. Combine conditions with `&` (and) and `|` (or), each in parentheses. Save the result as a **new** DataFrame (`df1 = df[...]`) so the original stays intact.
- `groupby`: `df.groupby("company")["fare"].mean()` = the average fare per company.

Market shares (class 2). Share of firm f = (firm f's trips) ÷ (all trips). There's no single "right" definition: trips, revenue or distance-weighted shares can differ. Always say which one you use.

Check AI code with synthetic data. Make fake data where you *know* the answer (e.g. three firms with 50/30/20% of trips), run the code, and confirm it returns 50/30/20. Plus **unit tests**: `assert` statements for things that must always be true ("shares sum to 100%", "row count > 0").

Three views of working with raw data (class 3).

1. Never touch the raw data: trust a codebook + AI. *Risk:* prompting well is itself a skill, and you may not notice errors.
2. Explore manually first, then guide the AI. *Risk:* the AI might spot the issues faster than you.
3. Do all the processing yourself. *Risk:* humans make mistakes too. The course practises all three.

Pipeline order depends on size (class 3).

- Small data: **Load → Clean → Split → Analyze → Summarize**.
- Big data (hundreds of millions of rows, many files): **Load → Clean → Analyze → Aggregate → Summarize**, processing file by file and combining the summaries. Never load billions of rows at once.
- The bigger or more complex the cleaning, the more it pays to **clean once and save** the cleaned data, rather than re-cleaning every time.
- Print the **row count after every step**, so you can see what each filter removed.

---

2. Summarizing data: missing values and outliers (class 4, Pierre Bodéré)

Why is it missing? The three mechanisms (learn the exam-box examples)

Mechanism	Meaning	Exam-box example	What to do
MCAR: missing completely at random	Missingness is pure chance. The rows you still have are still representative.	Exams placed randomly in 5 boxes; one box is lost.	Analyse the remaining data (you just lose precision).
MAR: missing at random	Missingness depends on something you **observe**.	Exams sorted alphabetically; the A–E box is lost.	Control for, or impute using, the observed variable (e.g. multiple imputation).
MNAR: missing not at random	Missingness depends on the **missing value itself** (even after accounting for what you observe).	Exams sorted by grade; the box with the top 20% is lost.	Hard: needs a model of why it's missing (selection model) or an instrument. Simple analysis is biased.

- Real MNAR examples: high earners refuse to report income; patients drop out of a drug trial *because* the drug isn't working for them.

- **You cannot test which mechanism applies.** You can only check whether the pattern is *consistent with* a story. It's always an assumption.
- **The preschool data:** enrollment is missing mostly for 1-star centers, because Pennsylvania only requires higher-rated centers to report. The professor called this **MNAR**: missingness is tied to quality, which is the very thing the research studies. Prices were more often missing in early years (older data is less complete).
- How to diagnose: make a 0/1 "is missing" flag and average it by group: `df.groupby("stars_rating")["enrol_missing"].mean()`.

Disguised missing values (common on this quiz and in our homeworks): missing data is often **coded**, not blank.

- NHAMCS emergency-room data (HW1): -9 = blank, -8 = unknown, -7 = not applicable.
- Class 9 experiment: spending = -999.
- NYC 311: "Unspecified", "N/A", "0 Unspecified", ZIP "00000", closed date 1900-01-01.

If you average a column that contains -999, the mean is wrong. Recode these to missing first.

"Not applicable" blanks: a field can be blank *by design* (e.g. taxi company for a noise complaint). Judge missingness only among the rows where the field applies.

Outliers

- **Find them**: `describe(percentiles=[.005, .25, .5, .75, .995])` shows the extreme tails. A preschool price of $2/day vs a 0.5th percentile of $15 looks wrong. $249/day could be a weekly price typed as daily, or a genuinely premium school.
- A single extreme point can **create or flip** a relationship in a regression or chart.
- **Don't drop automatically.** Ask:
  - Does the question need the tail? (A luxury-segment question needs the top end.)
  - Does the same firm keep reporting the same odd value? (That's systematic, not a typo.)
  - Do other columns confirm or contradict it?
- **Winsorize** (also called clip) instead of deleting: values below the 0.5th percentile are set *to* the 0.5th percentile, and values above the 99.5th are set to the 99.5th. You keep the row but tame its influence. `col.clip(lower=q005, upper=q995)`.
- **Use a natural cap** when one exists: enrollment can't exceed licensed capacity, so cap it at capacity.
- Always **check robustness**: do the results change with and without the outliers? (Class 9: three impossible spending values nearly doubled the estimated effect, from $8 to $14.)

Grouping and describing

- `groupby(...).mean()`, `value_counts(normalize=True)` for shares, `pd.qcut(x, 3, labels=["Low","Middle","High"])` for terciles (three equal-sized groups).
- **Key economic insight from class 4: sorting.** High-quality, high-price preschools sit in richer, more educated neighborhoods (accredited centers average ~$63k median income vs ~$56k). Product quality and customer type are **not randomly matched**. That matters later for demand estimation: you can't treat "who buys the premium product" as random.

---

3. Visualization (class 5 + Schwabish reading)

First principles.

- A good graph **reveals** the data rather than hiding it, and tells a story.
- Always label **units and scale**.
- **Proportionality**: the ink or area used should match the size of the number. The *lie factor* is when a small effect is drawn huge.
- **Data-ink ratio**: remove ink that carries no data (heavy gridlines, 3D effects, textures).
- "A graph is only as good as the underlying model of the world": solar radiation vs stock prices can look convincing and still be nonsense (a **spurious correlation**, with no mechanism).
- Stick to conventions (the y-axis increases upward; time runs left to right) unless there's a reason not to.
- **BS-detector triggers**: missing units, no sample sizes, no error bars, causal language without a design.

Chart types and when to use them.

Goal	Use	Watch out for
Distribution of one variable	**Histogram** (keeps counts)	Outliers squash everything into one bar: winsorize first
Compare distributions across groups	**Density / ridge plot** (each group's shape normalised)	Loses group sizes
Compact comparison of groups	**Box plot** (median, quartiles, whiskers, outlier dots)	Hides shape; fine for small samples
Relationship between two variables	**Scatter** (x = cause, y = effect) + fitted line, **coefficient and SE printed on the chart**	Dense clouds are hard to read
Conditional mean / nonlinearity	**Binscatter**: bin x, plot the mean of y in each bin	Hides outliers and bunching
Trends over time across groups of very different size	**Index to 100** in the base year, one common axis	Shared raw axis flattens small groups; free-scale facets hide relative size
Compare categories	**Bar chart**, sorted, **starting at zero**	A non-zero baseline exaggerates differences
Correlations among many variables	**Heatmap**	
Geography	**Choropleth** (shaded areas), points, **hexagon grids** (cleaner than odd-shaped areas)	Needs shapefiles (geopandas); per-capita vs totals

Schwabish's three rules: (1) show the data, (2) reduce the clutter, (3) integrate the text and the graph (label the lines directly, not in a distant legend).

His fixes, worth one sentence each on a quiz:

- **Clutterplot**: label only the few points you discuss.
- **3D charts**: never; the depth distorts values.
- **Spaghetti chart** (too many lines): use small multiples.
- **Pie charts**: people compare angles and areas badly; use bars or a slope chart instead.
- **Mixed encodings** (bars for one series, dots for another): encode both the same way.

Form × function 2×2: static vs interactive (form), explanatory vs exploratory (function). Most economics work is static + explanatory. Dashboards (like our 311 map) are interactive + exploratory: "overview first, zoom and filter, then details on demand."

Colour: avoid default palettes and use colourblind-safe ones. About 10% of people have some colour-vision deficiency.

---

4. Merging and big data (class 7)

Joins (merges). Combining two tables on a shared **key** column (e.g. customer_id).

Join	Keeps
Inner	Only rows whose key appears in **both** tables
Left	**All** rows of the left table; unmatched right-side columns become missing (NaN)
Right	All rows of the right table
Outer	All rows of both

Cardinality: how many rows match per key.

- **1:1**: unique on both sides.
- **1:n** (one customer, many orders): the result has **one row per order**, so it grows. Expected.
- **n:1** (many orders, one customer lookup): the result keeps the **left row count** *if* the right key is unique.
- **n:n**: duplicates on both sides give **every combination**, and the row count can explode.

Merge gone wrong (the slide example). You expect an n:1 merge of 6 orders to give 6 rows, but the customer table has customer_id 2 twice ("Bob" and "Robert"), so you get 7. **Always compare `df.shape` before and after a merge.**

- Unexpected **growth** means duplicate keys.
- Unexpected **shrinkage** means key mismatches (an inner join dropping rows).
- In pandas use `validate="many_to_one"` (errors if the right side isn't unique) and `indicator=True` (shows matched vs unmatched rows).
- Before merging, `assert` that the key is unique where it should be.
- Quick check for duplicates: number of unique keys vs number of rows.

Worked example (practice quiz Q3): 500 customers, 1,200 orders, 50 customers never ordered. A **left** merge (customers on the left) gives 450 customers × their orders = all 1,200 orders as rows, **plus** 50 rows for the customers with no orders. That's **1,250 rows**, and those 50 rows have **missing** order_id and amount. "Exactly 500 rows and no missing values" is false twice over.

Key formats often don't match. ACS census data uses `GISJOIN` (`G` + state + county + tract), while the taxi data uses a plain number (`17031980000.0`). You write a function to convert one to the other (or ask AI to, then verify). Other practical points:

- The ACS file needs `encoding="latin1"`.
- **Filter the big lookup table first** (all-US ACS → Illinois/Chicago only) before merging.
- Test on the first 50,000 rows.

What the taxi + ACS merge found.

- Raw fare vs neighborhood income: correlation ≈ 0, because trip distance swamps everything.
- Fare per mile (controls for distance): 0.16. Restricted to tracts with ≥10 trips (less noise): 0.27.
- Tips vs income ≈ 0.6, but only for **credit-card** trips: cash tips are recorded as $0, a measurement problem.

Tools for big data.

Tool	What it is	When
pandas	In-memory, single core, easiest to read; AI writes it well	Fine below ~10M rows or whatever fits comfortably in RAM. **Crashes** if the data is bigger than RAM
Dask	pandas-like syntax, splits data into chunks, runs in parallel, **lazy**	Larger-than-memory data; simple aggregates are fast; **quantiles/medians are slow** (they need all the data shuffled together)

Tool	What it is	When
DuckDB	SQL engine inside Python; streams from disk, very fast	Huge data, joins on many keys. Demo: 300 GB of airline data in **13 seconds**; pandas never finished
Polars	Rust-based, multi-threaded, pandas-like; can be lazy	Faster than pandas; use small number types (`int32`)

- **Lazy evaluation** = the tool builds the whole plan first and only runs it when you ask (`.compute()`, `.collect()`). It can then skip unneeded columns and rows. pandas is **eager**: each line runs immediately.
- **Parquet** = a compressed, column-based file format. You can read only the columns you need, so it's much smaller and faster than CSV or JSON. (311 lab: about 30 GB of JSON became about 1.5 GB of Parquet.)
- **Sampling** (a random or every-k-th subset) is a valid way to prototype, as long as it's representative. In our lab, a sample of January 2010 only was **not** representative of 12 years.

SQL basics. SQL is a language for querying tables; DuckDB runs it on DataFrames or files.

- `SELECT city, AVG(rooms) AS avg_rooms FROM listings WHERE country = 'USA' GROUP BY city ORDER BY avg_rooms DESC LIMIT 5`
- **`WHERE` filters rows *before* grouping; `HAVING` filters groups *after* aggregating** (`... GROUP BY year HAVING COUNT(id) > 100`).
- `IS NULL` / `IS NOT NULL` for missing values; `LIKE 'j%'` for text patterns; `IN (...)`; `BETWEEN a AND b`.

---

5. Regression (class 8): the core of the quiz

5.1 The model and the vocabulary

Simple linear regression: $y_i = \beta_0 + \beta_1 \cdot x_i + u_i$

- y = outcome (dependent variable); x = explanatory variable (regressor, "control"); i = one observation.
- β_0 = intercept (the value of y when x = 0); β_1 = slope (the change in y for a one-unit increase in x).
- u = **error term**: everything else that affects y that we don't observe (unobservables).
- "Simple" = one x. "Multiple" = several x's.

Linear means linear in the *parameters*, not a straight line.

- Linear (OLS works): $y = a + b \cdot x + c \cdot x^2 + u$; $y = a + b \cdot x1 \cdot x2 + u$; polynomials of any degree.
- Not linear: $y = a \cdot b \cdot x + u$ (parameters multiply each other); $y = a + b \cdot x \cdot u$ (a parameter multiplies the error).
- **Practice Q1 answer: False.** $y = \beta_0 + \beta_1 x + \beta_2 x^2 + u$ is linear in β. Just create a column x^2 and run OLS.

5.2 What OLS does

- **OLS (ordinary least squares)** picks $\hat{\beta}0$ and $\hat{\beta}1$ (the hat means "estimated from the data") to **minimise the sum of squared residuals**, $\Sigma(y - \hat{y})^2$.
- **Fitted value** $\hat{y} = \hat{\beta}0 + \hat{\beta}1 \cdot x$. **Residual** $\hat{u} = y - \hat{y}$, our estimate of the unobservable error.
- Why squared? It has a simple closed-form solution and nice statistical properties. An alternative is **LAD** (least absolute deviations, $\Sigma|\hat{u}|$), which is less sensitive to outliers.

Sums of squares and R^2.

- SST = $\Sigma(y - \bar{y})^2$, the total variation in y.

- SSR = $\Sigma(\hat{y} - \bar{y})^2$, the variation explained by the model.
- SSE = $\Sigma(\hat{u})^2$, the unexplained variation.
- **SST = SSR + SSE. R^2 = SSR/SST = 1 − SSE/SST**: the share of variation in y explained (between 0 and 1).

What R^2 is NOT (final quiz Q4: **False**):

- It is **not** a test of whether the model is correctly specified or causal. A model with $R^2 = 0.85$ can still have omitted variable bias, and one with $R^2 = 0.2$ can be a perfect randomized experiment.
- R^2 **can't be compared across different samples or outcomes**. The colleague used a separate sample.
- Adding variables never lowers R^2 *in the same sample*, so a higher R^2 can just mean more variables.

5.3 Reading coefficients, and units

- In a multiple regression, `β_k` = the change in y for a one-unit change in x_k **holding the other x's constant** ("ceteris paribus").
- **Scaling** (practice Q5: **True**):
  - If x is divided by 1,000 (dollars → thousands of dollars), its coefficient is **multiplied by 1,000**: 2.5 per dollar becomes 2,500 per thousand dollars.
  - If y is divided by 1,000, the coefficients are divided by 1,000.
  - t-statistics, p-values and R^2 do **not** change. Only the units change.
- **Dummy variable** d (0/1, e.g. treated vs control, male vs female): its coefficient = the **average difference in y between the two groups, holding x fixed**. That's an **intercept shift**.
  - Final quiz Q5 (**False**): $\hat{\beta}2 = 5$ on a dummy means the groups differ in **level** by 5. It says nothing about whether the **effect of x** differs. For that you need an **interaction**: `y = β0 + β1x + β2d + β3(x·d) + ε`, and test β3. The slope is β1 for d = 0 and β1 + β3 for d = 1.
- Logs (preview; useful for reading papers): with `ln(y)` on the left, β ≈ the % change in y per unit of x (×100). With `ln(y)` on `ln(x)`, β = the **elasticity** (% change in y per 1% change in x).

5.4 Uncertainty: standard errors, t, p and confidence intervals

- **Standard error (SE)** = how much $\hat{\beta}$ would bounce around if we redrew the sample. It shrinks roughly like $1/\sqrt{N}$: four times the data halves the SE.
- **t-statistic** = $\hat{\beta}$ / SE. **Significant at 5%** when $|t| > 1.96$, i.e. the 95% confidence interval excludes 0.
- **95% CI** ≈ $\hat{\beta} \pm 1.96 \cdot SE$. Interpretation: a range of values consistent with the data.
- **p-value** = if the true β were 0, the probability of seeing an estimate at least this far from 0. p = 0.03 means it's unlikely to be pure noise **given the model is right**.
- **Statistical significance ≠ causation ≠ importance** (practice Q6: **False**).
  - Significance says nothing about *why* x and y move together. Omitted variables, reverse causality or selection can all produce a significant β.
  - Causation needs a **research design**: randomization, or a credible argument that x is unrelated to the error.
  - A tiny, unimportant effect can be significant with huge N.
  - The AI only applied the threshold $p < 0.05$.

5.5 Nice properties, and what breaks them

- **Unbiased**: if you repeated the study many times, the $\hat{\beta}$'s would **average** to the true β.
- **Consistent**: as $N \to \infty$, $\hat{\beta}$ **converges** to the true β. (Class demo: the slope went from 1.23 at N = 5 to 1.98 at N = 10,000, truth 2.)
- These require the **key assumption: the error u is uncorrelated with x** (x is "exogenous").

- **Practice Q4 (False / Uncertain)**: 1.5 at N = 5,000 and 1.7 at N = 50,000 don't prove β > 1.5.
  - Each sample gives a noisy estimate. You can't infer the direction of the truth from two estimates without their **standard errors**.
  - Estimates don't move monotonically toward the truth.
  - If the model is **biased** (omitted variable bias), *both* numbers converge to the wrong value, so more data doesn't fix bias. It only makes you more confident in the wrong number.
- **Non-normal errors**: β̂ is still unbiased and SEs still work in large samples. Our simulation showed 95% CIs covering 94–96% of the time. Normality is **not** required for large-sample inference.

5.6 Endogeneity and omitted variable bias (OVB): the most tested idea

- **Endogeneity** = an x is **correlated with the error term u**. Then β̂ is biased and inconsistent.
- Three main sources (know all three):
  1. **Omitted variables.**
  2. **Measurement error** in x.
  3. **Simultaneity / reverse causality**, e.g. price and quantity determined together.
- **Omitted variable bias.** True model `y = β0 + β1x1 + β2x2 + u`, but you leave out x2. Then x2 sits inside the error. If x2 is **correlated with x1** and **affects y** (β2 ≠ 0), x1 is now correlated with the error, so β̂1 is biased.
  - Short formula (class version): `β1̂ → β1 + β2·δ`, where δ = the slope from regressing x2 on x1 (in class, `3 + 1.5ρ`).
  - **Direction of the bias = sign(β2) × sign(corr(x1, x2)):**

	x2 raises y (β2 > 0)	**x2 lowers y (β2 < 0)**
corr(x1, x2) > 0	**upward** bias	downward bias
corr(x1, x2) < 0	downward bias	**upward** bias

  - **No bias** if the omitted variable is uncorrelated with x1, *or* if it doesn't affect y.
  - Class demo: truth 3; including x2 gave 2.81; omitting x2 (corr 0.7, β2 = 1.5) gave **3.91**.
- **Final quiz Q1 (False)**: "x1 and x2 are correlated, therefore endogeneity."
  - Correlation **between regressors** is called **multicollinearity**. It's not endogeneity.
  - Including both is exactly how you *avoid* OVB. β̂1 is still unbiased; it's just less precise (bigger SEs), because the two x's carry overlapping information.
  - Endogeneity is about correlation with the **error**.
  - (Uncertain if you argue other variables are omitted, but the correlation itself is not the cause.)
- **Perfect multicollinearity**: one x is an exact linear combination of others, so OLS **can't** run. Classic case: the **dummy variable trap**. 24 hour-of-day dummies plus a constant: the dummies add up to the constant column. Fix: drop one category (it becomes the baseline).
- **Measurement error in x**: random noise in x pulls β̂ **toward zero** (**attenuation bias**). Class simulation: 2.0 → 1.6 → 1.0 → 0.6 as the noise grew; the multiplier = var(x) / (var(x) + var(noise)). Measurement error in **y** (random) only adds noise: no bias, bigger SEs.
- **Selection on y** (only observing cases above a threshold, e.g. only successful startups): biases β̂, usually toward zero. Class: 2.0 → 0.87 when keeping only y > 15.
- **Survey sampling** (final quiz Q6, **False**): surveying only flagship-store Saturday-afternoon shoppers is a **non-random (convenience) sample**. They differ from typical customers (busier store, weekend crowd, more engaged shoppers), so the estimate is biased for "all stores". Fix: random sampling across stores, days and times, or reweighting.

5.7 Heteroskedasticity

- = the **error variance differs across observations** (e.g. bigger errors for bigger x).
- $\hat{\beta}$ is **still unbiased**, but the **default SEs are wrong**, usually too small. Class simulation: SEs 18% too small, so "95%" intervals missed 11.7% of the time. Fix: **robust (HC1) standard errors**, which missed 5.5%.
- **Clustered SEs** (class 9): observations in the same group (store, school) share shocks, so they aren't independent. Cluster by group, otherwise you overstate precision.

5.8 Fixed effects (FE)

- **Categorical variables** (brand, store, hour, neighborhood) enter as **dummy variables**: one 0/1 column per category, **minus one** (the baseline is absorbed by the intercept, to avoid the dummy trap). `pd.get_dummies(df["store"], drop_first=True)`.
- Fixed effects **control for everything constant within the group**, observed or not (a store's location, clientele, manager). The coefficient on x then comes **only from variation within each group**.
- **Within estimator**: subtract each group's mean from y and x (`groupby("firm").transform(lambda v: v - v.mean())`) and regress the demeaned values. You get the same slope as with dummies, but you don't see the FE estimates. (With manual demeaning, the SEs need a degrees-of-freedom correction.)
- Examples from the slides:
  - 311: time to resolve ~ neighborhood FE.
  - Yelp: rating ~ cuisine FE.
  - TSA: passengers ~ hour FE + airport FE.
  - Shoe demand: brand FE, because brand is correlated with price.
- FE **remove** bias from omitted factors that are **constant within the group**. They do **not** fix omitted factors that **change within** the group over time.

5.9 Experiments

- **Randomization** makes treatment uncorrelated with everything else (on average), so $Purchase = \beta 0 + \beta 1 \cdot Treatment + \varepsilon$ gives an **unbiased causal effect**. $\hat{\beta}1$ = the difference in group means. Class: +$15 on a $50 base.
- **Balance checks**: compare treated vs control on pre-treatment characteristics (household size, tenure). They should look alike.
- **Class 9 free-delivery case** (know this story):
  - The offer went to 15% of customers in some stores and 90% in others, and high-share stores were stores whose customers already spent more.
  - The simple comparison gave **+$26**: biased upward by the store, an omitted variable.
  - **Store fixed effects** compare offered vs not-offered customers **within the same store**, which gave about **+$8** (95% CI $6.5–$9.9).
  - Finance needed +$15, so the answer was **no rollout**.
  - Randomization **within** stores made the within-store comparison valid.

5.10 Prediction vs causation

- **Prediction** asks "what will y be?", so any correlated feature helps (R^2 matters).
- **Causation** asks "what happens to y if *we change* x?", so we need x unrelated to the error (design matters).
- The same regression can be great for prediction and useless for causation (e.g. ice-cream sales predict drownings).

---

6. Validating results and reproducible analytics (class 9)

- **Never change the raw data.** Read it, clean a copy in code, and save the clean version as a **new** file. Anyone can rerun from scratch, and every cleaning decision is a line of code you can change. (Proof: compare a file "fingerprint" (hash) before and after the run.)
- **Synthetic data** with a known answer to test code (seed the random numbers for reproducibility: `np.random.seed(42)` makes "random" numbers identical each run).
- **Unit tests / asserts**:
  - Labels are all known (stop if an unexpected label appears).
  - IDs are unique.
  - The merge didn't change the row count.
  - Shares sum to 1.
- **Document every cleaning rule and its row count** (a decision log).
- **Automation with guardrails**: the same code must handle the next export ("wave 2" arrived with new labels, Offer/Holdout, and the pipeline correctly *stopped* instead of guessing).
- Data traps from the case:
  - **TEST accounts** (fake rows, all treated, $1 spends).
  - **Exact duplicates**.
  - **The same customer in both groups** (contaminated).
  - **-999** missing codes.
  - **Impossible values** ($14,890 vs a normal max of ~$400).
  - **Inconsistent labels** ("treat", "T", "1", "Treatment"; store "s09" vs "S09 ").
  - Each shortcut gave a different wrong answer: no cleaning **$5.6**, extremes kept **$14.0**, no store FE **$26**, correct **$8.2**.
- **Decision rule against a threshold, using the CI**:
  - Yes if the whole 95% CI is above the break-even.
  - No if it's entirely below.
  - Otherwise "inconclusive, collect more data".

---

7. End-to-end workflows: the 311 lab and quiz section 3 (class 10)

The 311 data: 28.2M requests, 2,824 JSON files, Jan 2010 – Nov 2021, about 64 GB if loaded into pandas.

Data-quality issues we found (good material for the "what data issue" sub-question):

- **Default timestamps**: 20% of closings at exactly midnight and about 1.9M at exactly noon. These are "date only" or default times, not real events. 33% of 2010–2014 creations are date-only (30–34% depending on the year).
- **1.04M closed in the same second they opened** (automatic closures); 261k–267k **closed before they were opened**; 5.9k **1900-01-01 placeholder** dates.
- **Mass administrative closures**: thousands closed in one second, years later. That's a clean-up, not service.
- **Disguised missing values** ("Unspecified" borough, "N/A"); fields **blank by design** for most complaint types.
- **Complaint labels**: 478 spellings for about 455 real types (HEATING vs HEAT/HOT WATER, upper/lower case, plurals), plus **hacking strings** typed into the web form.
- **Gaps** in collection (days with almost no records), and an incomplete final month.
- Time to close: **median 1.4 days but mean 23.6**, a long right tail. **Report medians** for skewed data.

The workflow-critique question (practice §2 and final §3): a model answer pattern

(A) Problems with "load 35M / 50M rows into one pandas DataFrame, then filter, compute, merge, group".

1. **Memory**: 50M rows × 10+ columns, with text and dates stored as strings, needs tens of GB, more than 16 GB of RAM. **Symptoms**: the kernel dies or restarts, "MemoryError", or the laptop freezes and swaps to disk (extremely slow).
2. **Wasted work**: loading everything and *then* filtering to 2023 reads and stores data you throw away. Every new column (`total_time`, `wait_time`) adds another full-length column to memory.
3. **Merge risks**: merging 50M rows with a 500k-row table is expensive. If `restaurant_id` **isn't unique** in the restaurant table (an n:n merge), rows **multiply** and memory explodes. Check uniqueness and the row count before and after.
4. **Data types**: dates read as text must be parsed, and missing or invalid dates (NaT) make durations missing or negative.
5. **Output**: Excel holds at most 1,048,576 rows, so a large driver × cuisine table may not fit. Export a summary.
6. **Statistics**: the **mean** is dragged by outliers (1,000-day "resolution times"), so use the median as well. Small groups give noisy averages.

(B) More efficient workflow.

1. **Read only needed columns** (`usecols`), with compact data types, and **filter while reading**: chunks, or DuckDB/Polars/Dask with lazy evaluation pushing the year filter down to the file read.
2. Compute durations **after** filtering. **Reduce the lookup table first** (only `restaurant_id, cuisine`, de-duplicated) and assert it's unique.
3. **Aggregate before merging where possible**: group by driver and restaurant first (far fewer rows), then merge cuisine onto the small table.
4. Store intermediate results as **Parquet**.

The principle: shrink the data as early as possible (select columns, filter rows, aggregate) and **push work to tools that stream from disk** instead of holding everything in memory.

(C) "Driver #4521 averages 8 minutes vs 35 for the company" / "some complaint types average 1,000+ days": what to check.

- **Small sample**: how many deliveries does the driver have? With 3 orders, the mean is noise.
- **Timestamp errors**:
  - delivery_time before or equal to pickup/order time (negative or zero durations)
  - wrong time zones
  - placeholder dates (1900, or a default like 9999)
  - date-only stamps
  - records closed in bulk years later
- **Duplicates or test records**; mis-assigned driver_id.
- **Selection / composition**: the driver may get short, nearby orders or one restaurant (compare like with like: same restaurant or distance). Cancelled orders may be logged with tiny times.
- **How to check**:
  - count records per driver or type
  - plot the distribution of durations
  - count negatives and zeros
  - look at min/max dates
  - inspect raw rows

- compare **median vs mean**
- re-run excluding suspicious records
- check status fields (e.g. still-open tickets)

---

8. Readings (what each one is about; skim the introductions you haven't read)

> Items 1–4 and 7–9 are from the papers / our notes (7–9 verified against the PDFs; see `class11-reading-notes.md`, `class12-reading-notes-*.md`). Items 5–6 are summarised from general knowledge, so check their introductions.

1. **Bodéré, "Dynamic Spatial Competition in Early Education"** (class 4).
   - Pennsylvania preschools 2010–2018: STAR quality ratings 1–4, prices, enrollment, policy.
   - Four facts: (1) access is unequal (more and better centers near college-educated families); (2) centers in poor areas exit more and upgrade less; (3) quality upgrades raise enrollment *and* prices, more so in richer areas (pricing to local willingness to pay); (4) providers respond to subsidies (+$100/day ≈ +3 percentage points chance of upgrading).
   - Data: Freedom of Information Act data plus ACS demographics. Missing enrollment for STAR 1 by design.
2. **Schwabish, "An Economist's Guide to Visualizing Data"** (class 5): see §3.
3. **NYT Upshot, "Air Travel Is Already Back to Normal in Some Places"** (class 6; TSA data analysed by our professor).
   - Recovery depended on **airport type**: vacation and outdoor airports recovered, big hubs didn't, and Hawaii stayed down because of entry rules.
   - Mechanisms: trip purpose, closed urban attractions, local risk tolerance.
4. **NYC311 Monitoring Tool** (OSC report, class 10).
   - Counts by complaint type and neighborhood.
   - Our critique: pie charts; counts not resolution; per-resident not per-unit; dropped invalid ZIPs; can't be reproduced.
5. **Fréchette, Lizzeri & Salz (2019, AER), "Frictions in a Competitive, Regulated Market: Evidence from Taxis"** (class 2, intro).
   - NYC yellow cabs: entry is capped by **medallions**, and drivers must **search** for passengers (matching frictions).
   - Uses trip data and a dynamic model of drivers' entry and shift decisions to ask how removing the medallion cap or reducing search frictions would change the number of taxis, wait times and welfare.
6. **Buchholz (2022, REStud), "Spatial Equilibrium, Search Frictions and Dynamic Efficiency in the Taxi Industry"** (class 2, intro).
   - NYC taxis are spatially **mismatched** (too many in Manhattan, too few elsewhere) because regulated fares don't vary by location and drivers search without information.
   - Shows that smarter pricing or dispatch (as ride-hail apps do) improves efficiency.
7. **Adams & Williams (2019, AEJ: Micro), "Zone Pricing in Retail Oligopoly"** (class 11, intro + data).
   - Home Depot vs Lowe's: online prices for the same product (e.g. drywall) collected **store by store**. Retailers charge different prices in different "zones" depending on local competition.
   - Asks how zone pricing vs uniform pricing affects prices and consumers.
   - Data collected from retailer websites: this is the **web-scraping / API** application of class 11.
8. **Deb, Öry & Williams (2024, AER), "Aiming for the Goal: Contribution Dynamics of Crowdfunding"** (class 12; verified, full notes in `class12-reading-notes-kickstarter.md`).
   - Data: **all 42,462 Kickstarter campaigns** launched Mar 2017–Sep 2018, scraped **every 12 hours**; reward pledges ("investors") kept separate from **donations** (no reward). Donations = 14% of money raised; 42% of campaigns succeed (all-or-nothing).

- Facts: donations **fall 72% within 3 days after the goal is reached** (reward pledges only −37%); donations are 4.8% of revenue for campaigns that hit the goal early vs 34.3% for those hitting it in the last 3 days; **78% of deadline successes would have failed without last-minute donations**.
- Model: a donor who cares about success gives **over time as a costly signal**, which solves buyers' coordination problem. Counterfactual: ~16% succeed with dynamic donor signaling vs ~3% with none.
- Data processing application: repeated HTML snapshots (our Task 5).

9. **Quan & Williams (2018, RAND), "Product Variety, Across-Market Demand Heterogeneity, and the Value of Online Retail"** (class 12; verified, full notes in `class12-reading-notes-product-variety.md`).
   - Data: shoe sales from "a large online retailer" (unnamed in the paper; same kind of data as our HW2) by location.
   - Facts: tastes **differ a lot by place** (a market's own top 500 shoes = 67% of its revenue but only 7% of national revenue; boots in cold states, sandals in warm). Local data are mostly **zeros** (99.96% of products sell 0 at the zip-month level), so the authors use how often products sell zero across markets to measure taste differences.
   - Method warning: dropping zeros gives implausibly small price elasticities (−0.88); their model: −3.3 (men), −2.1 (women).
   - Result: the consumer gain from online variety is **$37.1M (5.8%), not $65.1M (10.6%)** when local tastes and store stocking are accounted for, i.e. ~45% lower. Ignoring heterogeneity **overstates** the value of the "long tail."

---

9. Classes 11 and 12: data collection and data processing (as actually taught)

> Full notes: `class11-slides.md`, `class12-slides.md`, `class12-2026-10-13.md`. These two classes are the most likely source of a **workflow-critique (Section 3)** question and of 1–2 True/False/Uncertain statements.

9.1 Class 11 (Oct 8): getting data from the internet

Four ways to get web data, easiest to hardest:

1. **Download a file directly** with `requests`.
   - Example: the EIA weekly gasoline prices Excel file.
   - `response = requests.get(url)`, then check `response.status_code == 200`.
     - 200 = OK, 403 = forbidden (often bot-blocking), 404 = not found, 429 = too many requests.
   - Read it with `pd.read_excel(BytesIO(response.content))`. Nothing is saved to disk.
   - To repeat it weekly, schedule the script (**cron** on Mac/Linux, Task Scheduler on Windows).
2. **Use an API** (application programming interface): the site's official "data door".
   - Example: the Treasury `debt_to_penny` API. You put the request in the URL (`?filter=record_date:gte:2024-01-01`) and get **JSON** back.
   - Things to watch:
     - an **API key** is often required (inside firms too);
     - **rate limits / throttling** (max N requests per minute);
     - **pagination** (results come in pages: `page[size]`, `page[number]`, so you must loop);
     - numbers arrive as **text** (`"34001127522066.83"`): convert them with `pd.to_numeric` before any math;
     - `verify=False` switches off security-certificate checking (a workaround, not good practice).
3. **Find the site's internal API** (the professor's procedure):

    - Open the browser's **Web Developer Tools → Network tab**.
    - Watch which requests the page makes for prices and inventory.
    - Call those directly. That's much lighter than loading whole pages, but **easier for the site to detect**, so strike a balance.
    - (Our Chipotle scrape: we found `services.chipotle.com` this way. Calls had to run from inside a real browser because of **Kasada** bot protection, the same idea as **Akamai** at Home Depot.)
4. **Drive a real browser from code**:
    - **Playwright** (Microsoft): a library that launches Chrome and handles network and cookies.
    - **agent-browser** (Vercel): a fast command-line tool that works well with AI agents.
    - Others: Claude in Chrome, Browser Use, Stagehand.

Legality (the professor: "not legal advice"):

- *hiQ v. LinkedIn*: scraping public data does **not** violate the anti-hacking law (CFAA).
- *X Corp. v. Bright Data* (2024): courts are skeptical that **terms of service alone** can ban scraping public content.
- *Compulife v. Newman* (2024): public data can still be off-limits if it is effectively a **trade secret**.
- Firms often sell data to an **aggregator** (e.g. Nielsen), which sells it back to competitors.

Practical points:

- Firms scrape competitors **and themselves** (to check the price consumers see matches the system).
- Sites change constantly, so code that works today may break tomorrow. **Monitor it**: the professor's scripts email him stats (pages collected, errors).

9.2 Class 12 (Oct 13): turning messy data into a table

The big idea: scraped data almost never arrives as rows × columns. You have to **convert** it: unwrap, decode, flatten, and turn text into numbers.

Pick the tool by the shape of the data:

What you have	Example	Tool
HTML page	product cards `<div data-testid="product-pod">`	**BeautifulSoup** (finds elements by tag/attribute)
JSON hidden inside the page	`window.__APOLLO_STATE__={...}`	find it, then `json` (skip HTML parsing!)
Nested JSON	Variants → 14SCREEN → Price	`json`, `pd.json_normalize`
Text with a wrapper	`jQuery123({"state":"NY"})`, cookie `bankplancode=...%3D802...`	**regex**, then `json` / `unquote`
Free text	"Medium 2-Topping Pan Pizza and 2-Liter Coke"	regex for easy fields, a model for the rest
PDF tables	TSA, 996 pages	Adobe Action → Excel, **pdfplumber**, OCR for images

Key terms in plain words:

- **BeautifulSoup**: reads HTML into a tree so you can say "find every product card, then its price."
- **Regular expression (regex)**: a pattern for finding text. `\$\s*(\d+)\.(\d{2})` means "a dollar sign, maybe spaces, digits, a dot, two digits." You don't need to write them by hand; give the AI an example. Regex can be slow, so use it sparingly.

- **Embedded JSON**: many sites send the raw data inside the page and build what you see with JavaScript. **Always ask "is there JSON I can grab directly?"** It's cleaner and less fragile than HTML.
- `glob`: lists every file matching a pattern (`glob.glob("*/*.cdRate.json")`).
    - It builds the **whole list in memory**. With millions of files, use a **lazy** version (`glob.iglob`): one name at a time.
- `multiprocessing`: run the same function on many files at once across CPU cores (a "pool of workers").
    - It only helps when the jobs are **independent** (one file each).
    - **Don't ask for more cores than you have.** Tune up gradually (4 → 8 → 16). Cluster nodes have 128.

Lessons from the five in-class tasks (good T/F/U and critique material):

1. **Inspect the structure before converting.**
    - The TSA PDF changed format at least 7 times over the years.
    - Most students failed to convert it by just telling the AI "convert this"; the best asked it to *describe the file first*.
    - Also ask "what is the best tool for this job?" (Adobe can beat Python for a one-off).
2. **Validate every extraction against something independent.**
    - Our 2026 Home Depot prices matched the scraper's audit table 100%.
    - Our TSA daily totals matched TSA's published ~2–2.7M passengers/day.
    - A fast regex matched the professor's BeautifulSoup function on a sample.
3. **Scraped data hides disguised missing values.**
    - **"$0.00"** prices: 213 Home Depot stores; the item wasn't sold there, it wasn't free.
    - Non-products on menus (St. Jude donations at Domino's).
    - Snapshots from years after a campaign ended.
4. **Report forms have blanks that mean "same as above."**
    - The PDF prints the date/airport only on the first row of each group, so you **fill down**.
5. **Irregular snapshots → a regular panel.**
    - Kickstarter pages were saved roughly, not exactly, every 12 hours, and some are missing.
    - To get one row per project per 12 hours:
        - build the fixed time grid;
        - carry the **last observation forward**, **never using a future snapshot** (that would "know the future");
        - flag the filled rows.
6. **Values can go down.**
    - Kickstarter pledges get **withdrawn**: one project reached 513% of its goal, then backers cut pledges by 90% with the backer count unchanged, and it **failed**.
    - Don't trust a single snapshot; use the full path.
    - Also, percent-of-goal is wild for tiny goals ($20 goal: one $25 pledge = +125%).
7. **Pricing structure results** (ties to Adams & Williams):
    - **Chase** sets CD rates in ~72 regions but has only **2 distinct prices** nationally (one promo CD).
    - **Domino's** (franchisees set prices) varies a lot: ~950 unique prices, ~30% range; sides and drinks vary most.
    - **Home Depot** uses **zone pricing**: ~80 prices across ~1,800 stores, about 3× from cheapest to dearest (2013). In 2026 the zones persist, slightly coarser and redrawn.
8. **Prompting habits the professor recommends:**
    - Have the AI keep a **journal/README** of how it scraped a site, so it can adapt when the site changes.
    - **Prefer JSON over HTML.**
    - Try `curl` → `requests` → a browser.
    - Use multiprocessing for big directories.
    - If one model refuses or fails on bot detection, **try another model**.

Likely quiz statements (answers in §12, items 26–33).

---

10. Insurance: the two later topics that appeared on last year's papers

10.1 Supply, demand and instruments (syllabus class 15; appears as Q2 on *both* papers)

- **Simultaneity**: price and quantity are set together where supply meets demand. Regressing quantity on price mixes the supply curve and the demand curve, so the slope is neither.
- An **instrument** z for price must be (1) **relevant**: it moves price; and (2) **excluded / exogenous**: it affects quantity *only through price* in the equation you're estimating (uncorrelated with that equation's error).
- **To estimate DEMAND you need something that shifts SUPPLY** (e.g. input costs, weather at the farm). Shifting supply moves the equilibrium **along the demand curve**, tracing it out.
- **To estimate SUPPLY you need a DEMAND shifter** (e.g. income, holidays), which traces out the supply curve.
- **Two-stage least squares (2SLS)**: (1) regress price on the instrument, keeping the predicted price; (2) regress quantity on predicted price.
- **Final Q2 (False)**: a supply shifter (input costs) identifies the **demand** slope α1, not the supply slope β1. It shifts supply along a fixed demand curve.
- **Practice Q2 (False)**: validity is **equation-specific**, and the answer is False whichever way you read the wording:
  - If z^D is a **demand shifter**: it's excluded from the supply equation, so it identifies **supply**. But it appears in the demand equation, so it can't be used to estimate demand.
  - If "valid instrument for demand" means "used to estimate demand" (i.e. it's a **supply shifter**): it belongs in the supply equation, so it's **not** excluded there and is invalid for estimating supply.
  - Either way, one variable can't serve both equations: each curve needs a shifter of the *other* curve. Say both readings in your answer.

10.2 Discrete choice and willingness to pay (syllabus class 18; Section 2 on both papers)

- Each person picks the option j with the highest **utility**: `u_ij = α·price_j + ϕ·feature_j + ξ_j + ε_ij`. Coefficients show how each feature changes utility. The **sign** says like/dislike.
- **Ratio of coefficients = willingness to pay (WTP)**: how much price someone would accept for one more unit of a feature = ϕ / (−α).
- **Outside option** = not choosing any of the listed options (e.g. living outside Jakarta). Without it, the model can only shift people *between* inside options, never in or out of the market. If something hits *every* inside option equally, only the outside option's share changes.
- **ξ (xi)** = unobserved quality of the option. It's correlated with price (better places cost more), so price is endogenous. Hence the IV estimation and fixed effects in those papers.

Practice §2 (Jakarta). Rent −0.032, flooding −0.490, amenities (distance in km) −0.110.

- **A.** Higher rent lowers utility: each extra USD per m² per year of rent lowers utility by 0.032, holding flood risk and amenities fixed. People dislike paying more. (Use it to price other features.)
- **B.** No. The variable is **distance** to amenities, and the negative coefficient means people dislike being *far* from schools, clinics and rail, i.e. they **like** amenities.
- **C.** 5 km farther lowers utility by 5 × 0.110 = **0.55**. In money terms that's 0.55 / 0.032 ≈ **$17 per m² per year**: rent would have to fall about $17/m²/yr to compensate. Fewer people choose that area.
- **D.** Uniform sea-level rise makes **every** Jakarta location worse, so people shift **toward the outside option** (leaving Jakarta). Within Jakarta they move toward **less-flooded** areas if the increase isn't equal everywhere. Flood WTP:

0.49 / 0.032 ≈ \$15 per m² per year per metre of flooding.

2025 final §2 (Bay Area). WTP per 10%: white share +\$1,558; crime −\$586; ozone −\$296.

- **A.** Neighbourhood A: 10% less crime is worth **+\$585.56**; 5% fewer white residents = half of 10%, so 0.5 × −\$1,558.20 = **−\$779.10**. Net = **−\$193.54**, so the household prefers **B** (on these two attributes alone).
- **B.** Without an outside option, any change that makes the whole Bay Area better or worse couldn't change how many people live there. Total demand would be fixed and only redistributed. The outside option lets the model capture people **moving in or out**, which also anchors the level of utility.
- **C.** Mostly no change in *relative* sorting if the cut affects all neighbourhoods identically in utility. Everyone gains the same, so the ranking of neighbourhoods is unchanged, and mainly the outside-option share falls (more people stay or move in). But a **20% proportional** cut shrinks *absolute* crime gaps (high-crime areas gain more in levels), so some households would re-sort toward previously high-crime, cheaper areas. Prices would also adjust in equilibrium. A nuanced "it depends, here's why" answer scores best.

---

11. Worked answers to both practice papers (Section 1)

#	Practice quiz (Fall 2025)	Answer	One-line reason
1	Quadratic model can't be estimated by regression	**False**	Linear in parameters; regress y on x and x^2 with OLS
2	A demand instrument is also valid for supply	**False / Uncertain**	Validity is equation-specific; each curve needs a shifter of the *other* curve (§10.1)
3	Left merge gives exactly 500 rows, no missing values	**False**	1:n gives 1,250 rows; the 50 non-buyers have missing order columns
4	Estimate rose 1.5→1.7, so true β > 1.5	**False**	Sampling noise; need SEs; bias doesn't shrink with N
5	Advertising in \$000s turns β = 2.5 into 2,500	**True**	Dividing x by 1,000 multiplies β by 1,000; fit unchanged
6	p = 0.03, so definitely causal	**False**	Significance ≠ causation; needs a design (randomization, no OVB)

#	Final quiz (Fall 2025)	Answer	One-line reason
1	x1, x2 correlated, so endogeneity	**False**	That's multicollinearity (bigger SEs); endogeneity = x correlated with the error
2	Supply-shifting instrument identifies the supply slope	**False**	It identifies **demand** (traces the demand curve)
3	`df.loc[df.salary > 100000]` always has < 1,000 rows	**False**	Could be all 1,000 (≤, not <); also NaN salaries drop out
4	R^2 0.85 > 0.60, so your model is better specified	**False**	R^2 isn't a specification test; different samples; OVB possible
5	Significant dummy, so the effect of x differs by group	**False**	A dummy shifts the intercept; you need an x·d interaction
6	Saturday flagship survey is unbiased for all stores	**False**	Convenience sample: selection bias

Section 2 answers: §10.2. Section 3 / §2-workflow answers: §7.

---

12. Self-test bank (cover the answers, say yours out loud first)

1. **T/F/U:** "Adding a variable to a regression can lower R^2." → **False** (same sample: R^2 never falls; adjusted R^2 can).

2. **T/F/U:** "If a variable is omitted but uncorrelated with the included regressor, the coefficient is biased." → **False** (no bias; only precision is affected).
3. **T/F/U:** "Heteroskedasticity biases OLS slopes." → **False** (the slopes are fine; the SEs are wrong; use robust SEs).
4. **T/F/U:** "Measurement error in x makes the coefficient larger." → **False** (attenuation toward zero).
5. **T/F/U:** "Fixed effects remove all omitted variable bias." → **False** (only factors constant within the group).
6. **T/F/U:** "Including all 7 day-of-week dummies and a constant is fine." → **False** (dummy trap: perfect multicollinearity; drop one).
7. **T/F/U:** "A left merge never changes the number of left-table rows." → **False** (duplicate keys on the right duplicate left rows; 1:n grows it).
8. **T/F/U:** "MAR data can be ignored safely." → **False/U** (you must condition on or impute with the observed variable that drives the missingness; only MCAR is ignorable apart from lost precision).
9. **T/F/U:** "You can test whether data are MCAR or MNAR." → **False** (assumption; you can only check consistency).
10. **T/F/U:** "Dropping outliers is always the right fix." → **False** (check whether they're real or needed; winsorize; robustness).
11. **T/F/U:** "Dask computes medians as easily as means." → **False** (quantiles need a full shuffle; slow).
12. **T/F/U:** "WHERE can filter on COUNT(*)." → **False** (that's HAVING, after grouping).
13. **T/F/U:** "A bar chart may start at 50 to show differences better." → **False** (bars must start at zero, otherwise it's misleading).
14. **T/F/U:** "Larger N always reduces bias." → **False** (it reduces variance; bias remains).
15. **T/F/U:** "A randomized experiment needs no control variables for an unbiased effect." → **True** (controls can add precision).
16. **T/F/U:** "The mean is the best KPI for time-to-close." → **False/U** (skewed with outliers and bad dates; the median is more robust).
17. **Short:** Why does winsorizing beat deleting? (It keeps the row and its other information, limits influence, and doesn't shrink the sample selectively.)
18. **Short:** The class 9 naive estimate was $26 but store FE gave $8. Explain the sign of the bias. (Omitted store: high-treatment stores have higher spending, so corr(treat, store) > 0 and store raises spending, giving **upward** bias.)
19. **Short:** Why does fare per mile correlate with income but raw fare doesn't? (Distance dominates raw fare; normalising by miles removes it.)
20. **Short:** Name three cleaning checks before trusting a merge. (Key uniqueness, shape before/after, match rate via the indicator.)
21. **Short:** When should you index lines to 100? (Groups of very different sizes; you want to compare growth.)
22. **Short:** What is lazy evaluation and why does it help? (The plan is built before running, so the tool reads only the needed rows and columns and runs in parallel; it avoids loading everything.)
23. **Short:** 311 shows 1.04M requests closed the same second they opened. What are they and what do you do? (Auto or administrative closures; exclude from response-time KPIs or report separately.)
24. **Calc (no calculator):** $\beta = 0.032$ on rent, 0.110 on distance. What rent cut offsets a 2 km longer distance? ($2 \times 0.110 / 0.032 = 0.22 / 0.032 \approx$ **$6.9 per m² per year**.)
25. **Calc:** $\hat{\beta} = 3$, SE = 1. Significant at 5%? ($t = 3 > 1.96$, yes; 95% CI ≈ [1.04, 4.96].)
26. **T/F/U:** "Scraped data can be analysed directly once downloaded." → **False** (it must be parsed and converted: unwrap JSONP, decode cookies, flatten nested JSON, turn "3.50%" / "$25.47" text into numbers).
27. **T/F/U:** "To extract data from a web page you must parse its HTML with BeautifulSoup." → **False/U** (many sites embed the data as JSON in the page or load it from an API; reading that JSON directly is cleaner and more robust).
28. **T/F/U:** "Using more CPU cores with multiprocessing always makes processing faster." → **False** (only for independent jobs; overhead and disk/network limits cap gains; asking for more cores than exist slows things down. Tune 4 → 8 → 16).

29. **T/F/U:** "`glob.glob` is a good way to list a directory with 50 million files." → **False/U** (it builds the whole list in memory; use a lazy iterator like `glob.iglob` / `os.scandir`).
30. **T/F/U:** "A store showing a price of $0.00 is the cheapest store." → **False** (disguised missing value: not sold or not priced there; treat it as missing and report how many).
31. **T/F/U:** "To build a 12-hour panel from irregular snapshots, fill each slot with the nearest snapshot." → **False/U** (nearest can be a *future* snapshot, i.e. look-ahead; use the last snapshot at or before the slot and flag filled rows).
32. **Short:** A Kickstarter project's pledged total falls from $2,565 to $271 while backers stay at 13. Data error? (Probably not: backers cut or cancelled pledges, or the platform voided them. Check the id matches the folder and the goal/currency are constant; flag it; don't use one snapshot as "funding".)
33. **Short:** Name two ways to validate a scraper's output. (Compare with an independent total, e.g. TSA published daily counts; re-extract a sample with a second method; match an audit table; check row counts and duplicates; eyeball a few raw pages.)

---

13. One-page summary (the night before)

- **OLS**: minimise Σ residuals². R^2 = SSR/SST = 1 − SSE/SST. "Linear" = linear in the parameters.
- **Coefficient**: Δy per unit of x, holding the other x's fixed. Rescaling x by 1/k multiplies β by k. Dummy = intercept shift; interaction = slope difference.
- **SE ∝ 1/√N**; t = $\hat{\beta}$/SE; significant if |t| > 1.96; CI = $\hat{\beta}$ ± 1.96·SE. Significance ≠ causation ≠ importance.
- **Unbiased** (right on average) and **consistent** (right as N → ∞) **iff x is uncorrelated with the error**.
- **Endogeneity sources**: omitted variables, measurement error, simultaneity. OVB sign = sign(β_omitted) × sign(corr). Correlated regressors = multicollinearity, not endogeneity.
- **Measurement error in x** → attenuation. **Selection on y** → bias. **Heteroskedasticity** → SEs wrong (use robust); **clusters** → cluster SEs.
- **FE** = dummies (drop one) or demeaning; removes group-constant confounders only.
- **Experiments**: randomize, check balance, difference in means = effect. Decide using the CI vs the threshold.
- **Missing data**: MCAR / MAR / MNAR (random box / A–E box / top-grades box); untestable; watch for coded missing values (-9, -999, "Unspecified", 1900).
- **Outliers**: describe the percentiles, winsorize or cap, don't auto-drop, check robustness.
- **Merges**: check key uniqueness, shape before/after, validate, indicator. 1:n grows, n:n explodes, mismatches shrink.
- **Big data**: pandas below ~10M rows and within RAM; otherwise DuckDB/Polars/Dask, Parquet, lazy evaluation. Filter/select early, aggregate before merging. WHERE vs HAVING.
- **Graphs**: show the data, cut clutter, integrate text; bars from zero; no pies or 3D; index to 100; binscatter; coefficient (SE) printed on the chart.
- **Web data (cl. 11–12)**: file → `requests`; API → JSON (keys, rate limits, pagination, text→numbers); internal API via DevTools; browser automation (Playwright) when blocked. Look for **embedded JSON before HTML**; regex for patterns; glob (eager list; lazy for huge dirs) + multiprocessing (independent files, don't oversubscribe). **Inspect structure first, validate against an independent source**, watch $0 / placeholder values, build panels without look-ahead.
- **Always**: units, sample sizes, uncertainty, and the word that makes a T/F statement false.